

Database Description for Protocol Zone

This document provides a detailed explanation of the database schema used in the Protocol Zone application. The database is built using SQLite, a lightweight relational database that stores all data in a single file (habits.db). The schema consists of three main tables: users, trackers, and checkin_logs. These tables are designed to support user authentication, habit/goal tracking, and daily check-in logging, enabling efficient data storage and retrieval for analytics like streak calculations.

The database uses foreign key relationships to maintain data integrity, with cascading deletes to remove related records when a user or tracker is deleted. Data types are kept simple (TEXT for strings, INTEGER for numbers and IDs) to ensure compatibility and ease of use.

Described as:

```
DATABASE_PATH = os.path.join(PROJECT_FOLDER, 'habits.db')
```

Special assignments:

```
TRACKER_TYPES = ('subject', 'habit', 'goal')
```

```
CHECKIN_STATUSES = ('clean', 'slipped', 'unsure')
```

```
MIN_GOAL_DAYS = 1
```

```
MAX_GOAL_DAYS = 254
```

```
DEFAULT_GOAL_DAYS = 21
```

TABLES:

- 1) users
- 2) trackers
- 3) checkin_logs

USERS TABLE:

This table stores user account information for authentication and personalization.

- id: INTEGER PRIMARY KEY AUTOINCREMENT - Unique identifier for each user (auto-generated).

- username: TEXT NOT NULL UNIQUE - User's chosen display name (must be unique across all users).
- email: TEXT NOT NULL UNIQUE - User's email address for login (must be unique and used as the primary login credential).
- password_hash: TEXT NOT NULL - Hashed version of the user's password for secure storage (uses SHA-256 hashing).
- created_at: TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP - Timestamp when the user account was created (in YYYY-MM-DD HH:MM:SS format).

TRACKERS TABLE:

This table stores details about each habit, goal, or subject tracker created by users. Each user can have up to three trackers.

- id: INTEGER PRIMARY KEY AUTOINCREMENT - Unique identifier for each tracker (auto-generated).
- user_id: INTEGER NOT NULL - Foreign key linking to the users table (references users.id); ensures trackers belong to a specific user
- tracker_name: TEXT NOT NULL - Name of the tracker (e.g., "Morning Run" or "Math Study").
- tracker_type: TEXT NOT NULL CHECK(tracker_type IN ('subject', 'habit', 'goal')) - Type of tracker (limited to predefined values for consistency).
- quote_text: TEXT NOT NULL DEFAULT " " - Optional motivational quote associated with the tracker (defaults to empty string).
- goal_days: INTEGER NOT NULL CHECK(goal_days BETWEEN 1 AND 254) - Target number of days for the tracker (must be between 1 and 254).
- start_date: TEXT NOT NULL - Start date of the tracker in YYYY-MM-DD format (when tracking begins).
- created_at: TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP - Timestamp when the tracker was created.
- updated_at: TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP - Timestamp when the tracker was last updated.

CHECKIN_LOGS TABLE:

This table logs daily check-ins for each tracker, storing the status of each day to calculate streaks and progress.

- **id:** INTEGER PRIMARY KEY AUTOINCREMENT - Unique identifier for each check-in log entry (auto-generated).
- **tracker_id:** INTEGER NOT NULL - Foreign key linking to the trackers table (references trackers.id); ensures logs belong to a specific tracker.
- **date:** TEXT NOT NULL - Date of the check-in in YYYY-MM-DD format.
- **status:** TEXT NOT NULL CHECK(status IN ('clean', 'slipped', 'unsure')) - Status of the check-in (clean = success, slipped = failure, unsure = no final answer).
- **timestamp:** INTEGER NOT NULL - Unix timestamp (seconds since epoch) when the check-in was recorded.

Relationships:

users.id → trackers.user_id (one-to-many: one user can have multiple trackers); trackers.id → checkin_logs.tracker_id (one-to-many: one tracker can have multiple logs).

Constraints:

Unique constraints prevent duplicate check-ins per tracker per date. Check constraints enforce valid values for tracker_type, goal_days, and status.

Data Flow:

User inputs (e.g., via forms) are validated in the Flask app and inserted into these tables. Queries retrieve data for dashboard display, streak graphs, and milestones.

Migration:

The database includes logic for schema updates (e.g., adding new columns) to handle version changes without data loss